

dilemma

Codebase and Runtime Architecture Guide

Generated from the checked-in repository

September 10, 2026

Abstract

This guide explains the complete checked-in implementation of *dilemma*: a small browser game in which players answer moral or hypothetical questions, compare group choices, and earn points for matching the most common answer. It starts with the architecture and the user journeys, then describes persistence, security, every named application and support function, the test suite, and the generated catalog. Code excerpts use the `lstlisting` package and are deliberately short so that they remain readable in the compiled PDF.

Repository snapshot. The current `dilemmas.json` contains 40 entries and 149 options. `Claude.md` describes a planned catalog of 53 entries, so this document treats the checked-in files as authoritative and calls out the difference where it matters.

Contents

1	Executive overview	3
1.1	Top-down architecture	3
1.2	The central control loop	4
2	Repository map	4
3	User journeys	5
3.1	Singleplayer	5
3.2	Authentication and theme	5
3.3	Room creation and joining	5
3.4	Multiplayer round lifecycle	6
3.5	Answer and scoring flow	6
4	Data model and persistence	6
4.1	Local browser state	6
4.2	Firestore paths	7
4.3	History merge and idempotency	7
5	Security model	7

6	Function reference: browser runtime	8
6.1	js/app.js: DOM and state helpers	8
6.2	js/app.js: room and answer rendering	9
6.3	js/app.js: event callbacks	10
7	Function reference: Firebase and multiplayer	10
7.1	js/firebase.js	10
7.2	js/multiplayer.js: references and identity	10
7.3	js/multiplayer.js: room lifecycle	11
7.4	js/multiplayer.js: answer and score operations	11
8	Function reference: content, history, and pure logic	12
8.1	js/catalog.js	12
8.2	js/history.js	12
8.3	js/selection.js	12
8.4	js/scoring.js	13
9	Function reference: tools, rules, and tests	13
9.1	tools/write-config.mjs	13
9.2	Catalog generator	13
9.3	firestore.rules	14
9.4	Tests and test helpers	14
10	Markup and styling	15
10.1	HTML view structure	15
10.2	CSS behavior	15
11	Build, run, and verification	15
11.1	Configuration	15
11.2	Commands	15
11.3	Verification checklist	16
12	Notable implementation decisions	16
13	Summary	17

1 Executive overview

dilemma is a static, module-based web application. The browser loads one HTML shell, one stylesheet, and an ES module entry point. The entry point owns the user interface state machine and delegates domain work to small modules:

- `catalog.js` loads and indexes the JSON content.
- `selection.js` chooses fresh or least-recently-seen dilemmas.
- `scoring.js` tallies answers and applies the majority/tie rule.
- `history.js` stores private per-user history locally or in Firestore.
- `multiplayer.js` contains all room and round reads and writes.
- `firebase.js` initializes Firebase and re-exports the SDK pieces used elsewhere.

The most important design choice is the separation between **orchestration** and **pure helpers**. `app.js` reacts to user actions and live snapshots. Selection and scoring helpers are deterministic apart from an injected random function, which makes them easy to test. Firestore rules provide the final authority over who may read or write room state.

1.1 Top-down architecture

The runtime can be read from top to bottom as follows:

Layer	Responsibility and main files
Browser shell	<code>index.html</code> defines the home, auth, room, and singleplayer views. <code>styles.css</code> controls the monochrome Times New Roman UI, responsive layout, focus states, and dark mode variables.
UI coordinator	<code>js/app.js</code> attaches event handlers, maintains the single mutable <code>state</code> object, subscribes to Firestore, runs small reconciliation ticks, and renders the current view.
Game domain	<code>selection.js</code> handles repeat avoidance; <code>scoring.js</code> handles counts and points; <code>catalog.js</code> supplies prompt text, options, and source metadata.
Persistence	<code>history.js</code> owns <code>localStorage</code> and private account history. <code>multiplayer.js</code> owns rooms, players, round documents, choices, submission markers, candidate counts, and score transactions.
Backend boundary	<code>firebase.js</code> initializes Auth and Firestore only when configuration is present. <code>firestore.rules</code> limits reads and writes by identity, room membership, host status, and round phase.
Content and verification	<code>tools/generate_dilemmas.py</code> generates the catalog; <code>tools/write-config.mjs</code> generates <code>js/config.js</code> ; <code>test/</code> covers pure logic, browser smoke behavior, and Firestore rules.

1.2 The central control loop

The application uses live snapshots rather than a framework store. Whenever a snapshot or local action changes state, `update` runs four side-effect checks, computes a compact render signature, and renders only when the signature changed:

Listing 1: The reconciliation loop in `app.js`

```

1 function update() {
2   hostTick();
3   contributeTick();
4   historyTick();
5   scoreTick();
6   const signature = JSON.stringify([
7     state.room, state.roundNumber, state.contrib.length, state.round,
8     state.myChoice, state.players.map((p) => [p.uid, p.name, p.score]),
9     state.submitted.length,
10    state.choices.map((v) => [v.player, v.optionIndex]),
11  ]);
12  if (signature === state.signature) return;
13  state.signature = signature;
14  renderRoom();
15 }
```

This is a lightweight state machine. Snapshots update facts; the ticks advance work that is safe to retry; the renderer derives the DOM from the latest facts. The signature prevents a Firestore update that does not affect visible content from rebuilding controls and losing focus.

2 Repository map

Path	Role
<code>index.html</code>	Static DOM shell. Holds all four views and stable element IDs used by <code>app.js</code> .
<code>styles.css</code>	Presentation layer: theme variables, centered landing page, forms, option cards, responsive breakpoints, and reduced-motion support.
<code>js/app.js</code>	Browser entry point and UI orchestrator.
<code>js/firebase.js</code>	Firebase SDK imports, conditional initialization, and re-exports.
<code>js/catalog.js</code>	Top-level JSON fetch plus lookup functions.
<code>js/history.js</code>	Private history adapter for <code>localStorage</code> and Firestore.
<code>js/multiplayer.js</code>	Firestore data-access and room lifecycle operations.
<code>js/scoring.js</code>	Pure choice-count and point functions.
<code>js/selection.js</code>	Pure shuffle, freshness, candidate, and room-selection functions.
<code>dilemmas.json</code>	Generated runtime catalog; 40 entries in the current snapshot.
<code>firestore.rules</code>	Firestore authorization and field/phase validation.
<code>tools/generate_dilemmas.py</code>	Source-of-truth Python catalog builder and validator.
<code>tools/write-config.mjs</code>	Converts environment variables into browser-readable Firebase config.
<code>test/scoring.test.mjs</code>	Majority, tie, invalid, and empty-choice tests.
<code>test/selection.test.mjs</code>	Catalog shape, repeat avoidance, fallback, candidate, and cycling tests.
<code>test/smoke.test.mjs</code>	Headless browser checks for page behavior and console errors.
<code>test/rules.test.mjs</code>	Firestore emulator tests for room, history, pool, choice, and score boundaries.
<code>firebase.json</code>	Firebase CLI emulator/project configuration.
<code>package.json</code>	npm scripts and runtime/dev dependencies.

Path	Role
README.md	Short public-facing description and play link.
Claude.md	Product/MVP requirements and verification checklist.

3 User journeys

3.1 Singleplayer

Singleplayer does not require Firebase. Clicking **Singleplayer** calls `startSingleplayer`, which asks `pickSingleplayerQueue` for a queue as large as the catalog. The first item renders immediately. Rendering calls `recordSeen`, so a question is recorded when shown rather than when the player submits an answer. The form requires a selected radio button; submission advances the in-memory queue and renders the next entry.

The queue policy is:

1. Shuffle all IDs missing from the user's history.
2. If enough unseen entries exist, return only those entries.
3. If not, append previously seen IDs ordered from oldest to newest.
4. When the queue is exhausted, rebuild it with the current history.

This means repeat avoidance is a preference, not a hard availability requirement. A user who has seen the entire catalog can continue playing.

3.2 Authentication and theme

The sign-in view supports email/password sign-in, sign-up with an optional display name, sign-out, and passwordless email-link sign-in. Firebase Auth state drives the signed-in and signed-out subviews. A pending email-link address is stored under `moral-dilemma:email-link`; the address is removed after success or after an invalid/expired link. The URL is cleaned with `history.replaceState` after successful completion.

The theme toggle stores `light` or `dark` in `localStorage`. `applyTheme` changes `document.documentElement.dataset.theme`; CSS variables then switch the background, text, border, and soft-panel colors.

3.3 Room creation and joining

Room play requires a signed-in user. The create form accepts 1 through 20 rounds. `createRoom` makes a short random code, checks for a collision, writes the room document in `lobby` state, then adds the host as the first player. Joining reads the room, rejects missing or already-started rooms, and writes the joining user's own player document.

The lobby shows all current players. Only the host sees the rounds control and start button. The start button remains disabled until at least two players are present.

3.4 Multiplayer round lifecycle

State	What happens
lobby	Players can join. The host chooses the positive round count.
selecting	The host publishes candidates. Each player contributes only counts for candidates present in their own private history.
playing, answer	The host selects the least-seen candidates, starts round 1, and every player submits one option. Choices are hidden from other players.
playing, results	Once every player has a submission marker, the host changes the round to results. All players can read choices, compute percentages, and record their own score.
playing, next round	The host creates the next round, or moves the room to finished when the last round has been scored.
finished	Everyone sees the final scoreboard sorted by cumulative score.

The selection stage is privacy-preserving. A player never uploads their full history. They upload increments to shared per-candidate counts and a single contribution marker. The host then chooses the candidates with the fewest seen-player counts; ties are randomized. If there are too few candidates to fill all rounds, the chosen list cycles.

3.5 Answer and scoring flow

Each answer is written as two documents in a single batch: the player's private `choices/{uid}` document and the public `submitted/{uid}` marker. Rules allow only one create, so a player cannot change an answer after submitting. The host watches submission markers and opens results when their count reaches the player count.

The scoring rule is intentionally simple:

Listing 2: The majority and tie rule in `scoring.js`

```

1 export function pointsForChoice(optionIndex, choices) {
2   if (!Number.isInteger(optionIndex)) return 0;
3   const counts = tallyChoices(choices);
4   if (counts.size === 0) return 0;
5   const highest = Math.max(...counts.values());
6   const winners = [...counts.values()]
7     .filter((count) => count === highest).length;
8   return winners > 1 || counts.get(optionIndex) === highest ? 1 : 0;
9 }

```

Every player computes only their own points and writes only their own player document. `recordScore` uses a transaction and reconstructs the cumulative score from `roundScores`, so a retry does not double-add a point.

4 Data model and persistence

4.1 Local browser state

The app uses two `localStorage` keys:

- `moral-dilemma:history`: JSON object mapping dilemma ID to a millisecond timestamp for signed-out history.
- `moral-dilemma:email-link`: the pending email address for passwordless sign-in completion.

The large runtime state object in `app.js` is not persisted as one blob. It is a cache of the current user, room, players, round, choices, submitted markers, selection contributions, and idempotency guards. Firestore snapshots repopulate it after reconnects.

4.2 Firestore paths

Path	Fields and purpose
<code>users/{uid}/history/{dilemmaId}</code>	<code>dilemmaId</code> , <code>seenAt</code> . Private history keyed by the stable catalog ID.
<code>rooms/{roomId}</code>	<code>hostUid</code> , <code>status</code> , <code>totalRounds</code> , <code>currentRound</code> , <code>candidates</code> , <code>dilemmaIds</code> , and <code>createdAt</code> . Shared room state.
<code>rooms/{roomId}/players/{uid}</code>	<code>name</code> , <code>score</code> , <code>roundScores</code> , and <code>joinedAt</code> . One document per player.
<code>rooms/{roomId}/pool/{dilemmaId}</code>	A shared count of players who have seen the candidate.
<code>rooms/{roomId}/contrib/{uid}</code>	A marker showing that a player completed the selection contribution.
<code>rooms/{roomId}/rounds/{n}</code>	<code>dilemmaId</code> , copied prompt, copied options, and <code>phase</code> .
<code>.../rounds/{n}/choices/{uid}</code>	One private answer document containing <code>optionIndex</code> ; readable to players only after results.
<code>.../rounds/{n}/submitted/{uid}</code>	One public marker per submitted answer, used by the host as a completion count.

Round documents copy the prompt and options at start time. That makes an in-progress room stable even if the catalog file changes later. The stable catalog ID remains available for source attribution and history recording.

4.3 History merge and idempotency

On initial load, the app reads local history. When a user signs in, it loads the account's Firestore history, writes any local-only IDs into that account, removes the local key, and uses the merged map from then on. `recordSeen` updates the in-memory map before the remote write; if the write fails, it removes the optimistic entry.

The remote record uses the dilemma ID as the document ID and checks for an existing document before writing. This makes reconnects harmless: seeing the same dilemma again does not create duplicate records or reorder the timestamp.

5 Security model

`firestore.rules` is not just a deployment detail. It encodes the invariants the client assumes:

- Every room read requires authentication; collection listing is disabled.

- Room creation requires the creator to be the host, a lobby status, round zero, and a positive integer round count.
- Only the host may update the room or create/update round documents. The host ID cannot be transferred during an update.
- A player may create, update, or delete only their own player document.
- Candidate pool documents may change only during selection, and each update must increment the prior count by exactly one.
- A player may create only their own choice, only during the answer phase, only once, and only with an in-range integer option index.
- Choice documents become readable to players only after the round leaves the answer phase.
- Submission markers are one-time creates and are readable while the round is active.
- User history can be read and written only under the authenticated user's own UID.

The key privacy transition is small but important:

Listing 3: The choice visibility boundary in firestore.rules

```

1 match /choices/{uid} {
2   allow read: if isPlayer(roomId) && roundPhase(roomId, roundId) != 'answer';
3   allow create: if isPlayer(roomId) && uid == request.auth.uid
4     && roundPhase(roomId, roundId) == 'answer'
5     && request.resource.data.optionIndex is int
6     && !exists(/databases/$(database)/documents/rooms/$(roomId)
7       /rounds/$(roundId)/submitted/$(request.auth.uid));
8   allow update, delete: if false;
9 }

```

The client hides choices before results for usability; the rules enforce the same boundary for security.

6 Function reference: browser runtime

This section lists every named or exported function in the application modules. Anonymous event callbacks are described with the event or subscription that owns them.

6.1 js/app.js: DOM and state helpers

Function	Behavior
<code>\$(id)</code>	Shorthand for <code>document.getElementById</code> . Used for stable controls and view containers.
<code>el(tag, text)</code>	Creates a DOM node and, when supplied, assigns safe text content instead of parsing HTML.
<code>applyTheme(theme)</code>	Sets the root data attribute, updates the toggle label, and persists the theme when <code>localStorage</code> is available.
<code>showView(name)</code>	Hides every <code>.view</code> , shows <code>view-name</code> , and mirrors the view name on <code>body.dataset.view</code> .
<code>setError(id, message)</code>	Writes an error message and toggles the target's hidden state. Empty text clears it.

Function	Behavior
<code>stopRound()</code>	Unsubscribes round listeners and clears round-level state.
<code>stopRoom()</code>	Stops the round, unsubscribes room listeners, and clears room-level state.
<code>recordSeen(dilemmaId)</code>	Idempotently records a shown dilemma in the signed-in account or local history, with rollback on failure.
<code>requireAuth()</code>	Guards room actions; reports configuration problems or redirects signed-out users to the auth view.
<code>setEmailLinkStatus(message)</code>	Shows or hides the status paragraph for passwordless sign-in.
<code>emailLinkSettings()</code>	Builds Firebase's in-app email-link settings using the current origin and path.
<code>completeEmailLink(email)</code>	Validates the address, completes Firebase email-link sign-in, cleans the URL, and reports invalid/expired links.
<code>enterRoom(code)</code>	Resets prior room state, subscribes to the room, contribution, and player collections, and starts round subscriptions when the room advances.
<code>roomFailed(error)</code>	Places a Firestore subscription error in the room error area.
<code>watchRound(code, n)</code>	Subscribes to the round and submission marker collections; after results begin, also loads all choices and the current user's choice.
<code>isHost()</code>	Returns whether the current authenticated user owns the active room.
<code>hostTick()</code>	Host-only reconciliation: finalizes selection after all contributions, then opens results after all players submit.
<code>scoreTick()</code>	Waits for complete local results and records the current player's one-round point exactly once.
<code>contributeTick()</code>	During selection, sends this user's seen counts once for the current candidate list.
<code>historyTick()</code>	Records the current round's dilemma once per round.
<code>update()</code>	Runs the side-effect ticks, compares the render signature, and triggers room rendering only after meaningful changes.
<code>renderRoom()</code>	Thin rendering entry point that delegates to <code>renderRoomBody</code> .
<code>renderRoomBody()</code>	Selects lobby, selecting, finished, answer, results, or loading output from room status and round phase.

6.2 js/app.js: room and answer rendering

Function	Behavior
<code>renderLobby(body, room, players)</code>	Lists players, shows the host's round-count control, enables start only for two or more players, and proposes candidates.
<code>dilemmaBlock(round)</code>	Builds the displayed prompt plus its source attribution. It prefers the copied prompt in the round document and falls back to the catalog.
<code>sourceLine(source)</code>	Creates a safe linked source line and labels inspiration metadata separately from a direct source.
<code>renderChoiceAnswer(body, round, players)</code>	Renders the prompt, one radio option per catalog option, and a one-time submit button; after submission it shows the waiting count.

Function	Behavior
<code>renderChoiceResults(body, round, players)</code>	Tallies choices, computes percentages, reports the current player's points, renders scores, and gives only the host a next/finish button.
<code>renderScoreboard(body, players, title)</code>	Sorts a copy of the player list by cumulative score and renders an ordered list.
<code>startSingleplayer()</code>	Builds a full singleplayer queue from the current private history and renders its first entry.
<code>renderSingle()</code>	Rebuilds the singleplayer prompt and radio list, cycles the queue when exhausted, and records the displayed ID.

6.3 js/app.js: event callbacks

The module also registers anonymous callbacks for the theme button, back buttons, authentication forms, room create/join/leave buttons, singleplayer submission, and Firestore snapshots. Their common pattern is: prevent default browser navigation, clear stale errors, call a domain function, and show the error message if the promise rejects. Keeping these callbacks at the entry point means domain modules do not need to know about DOM nodes.

7 Function reference: Firebase and multiplayer

7.1 js/firebase.js

This file has no application-defined named function. It imports Firebase Auth and Firestore functions, declares `auth` and `db`, and conditionally initializes them when `isConfigured` is true. It then re-exports the SDK functions used by the rest of the app. This gives the app a single dependency boundary and allows an unconfigured checkout to keep singleplayer working.

7.2 js/multiplayer.js: references and identity

Function or value	Behavior
<code>randomCode()</code>	Uses four cryptographically random bytes and maps them into an alphabet that avoids ambiguous characters.
<code>roomRef(code)</code>	Returns the room document reference.
<code>playersRef(code)</code>	Returns the room's player collection reference.
<code>playerRef(code, uid)</code>	Returns one player's document reference.
<code>roundRef(code, n)</code>	Returns one round document reference.
<code>sub(code, n, name)</code>	Private collection helper for round subcollections.
<code>choicesRef(code, n)</code>	Returns the round choice collection.
<code>submittedRef(code, n)</code>	Returns the round submission-marker collection.
<code>choiceRef(code, n, uid)</code>	Returns one player's answer document; kept private to this module.
<code>poolRef(code)</code>	Returns the candidate count collection.
<code>contribRef(code)</code>	Returns the contribution marker collection.
<code>playerName(user)</code>	Uses Firebase display name, then email prefix, then the literal <code>player</code> .

7.3 js/multiplayer.js: room lifecycle

Function	Behavior
<code>createRoom(user, totalRounds)</code>	Tries up to five random codes, creates an empty lobby, adds the host, and returns the winning code.
<code>addPlayer(code, user)</code>	Writes the caller's player document with zero score and an empty round-score map.
<code>joinRoom(code, user)</code>	Verifies that the room exists and is still a lobby before calling <code>addPlayer</code> .
<code>leaveRoom(code, uid)</code>	Deletes the caller's player document. The room itself is not deleted.
<code>setTotalRounds(code, totalRounds)</code>	Updates the room's round count; rules ensure only the host can do this and the value stays positive.
<code>startRound(code, room, n)</code>	Creates a round document from the chosen stable dilemma ID, copied text, copied options, and answer phase.
<code>proposeSelection(code, totalRounds, catalogIds)</code>	Moves the room to selecting and publishes at least 20 candidates or four per round, whichever is larger.
<code>contribute(code, uid, candidates, history)</code>	Increments pool counts for locally seen candidates, then writes the caller's contribution marker last.
<code>finalizeSelection(code, room)</code>	Reads shared counts, chooses the least-seen IDs, writes them to the room, starts round 1, and moves to playing.
<code>nextRound(code, room)</code>	Finishes the game after the last round or starts the next round and advances <code>currentRound</code> .

7.4 js/multiplayer.js: answer and score operations

Function	Behavior
<code>myChoice(code, n, uid)</code>	Reads one answer document and returns its option index or null.
<code>submitChoice(code, n, uid, optionIndex)</code>	Preserves an existing answer; otherwise writes the answer and submission marker atomically in a batch.
<code>showResults(code, n)</code>	Changes the round phase from answer to results.
<code>recordScore(code, uid, roundNumber, points)</code>	Transactions the caller's player document, stores the per-round point, and recomputes the cumulative total.
<code>onSnapshot</code>	Re-export of the Firebase listener used by <code>app.js</code> .

Listing 4: Atomic answer submission in `multiplayer.js`

```

1 export async function submitChoice(code, n, uid, optionIndex) {
2   const existing = await myChoice(code, n, uid);
3   if (existing !== null && existing !== undefined) return existing;
4   const batch = writeBatch(db);
5   batch.set(choiceRef(code, n, uid), { optionIndex });
6   batch.set(doc(submittedRef(code, n), uid), { at: serverTimestamp() });
7   await batch.commit();
8   return optionIndex;
9 }

```

8 Function reference: content, history, and pure logic

8.1 js/catalog.js

The module performs a top-level fetch of `../dilemmas.json` relative to the module URL. It exposes the loaded array, an error value, and a stable ID array. A private `Map` makes lookups constant-time.

Function or value	Behavior
<code>CATALOG</code>	The loaded array, or an empty array if fetch/JSON parsing failed.
<code>catalogError</code>	The fetch or parse error, otherwise null.
<code>catalogIds</code>	Array of stable IDs derived from the loaded entries.
<code>entryFor(id)</code>	Returns the full entry or null.
<code>textFor(id)</code>	Returns prompt text or the empty string.
<code>optionsFor(id)</code>	Returns the option array or an empty array.
<code>sourceFor(id)</code>	Returns source metadata or null.

8.2 js/history.js

Function	Behavior
<code>historyRef(uid, dilemmaId)</code>	Builds the private Firestore history document path.
<code>readLocal()</code>	Parses the local history object and returns an empty object on missing or malformed storage.
<code>writeLocal(entries)</code>	Serializes local history; storage failures are ignored so gameplay can continue.
<code>localHistory()</code>	Converts local entries to a <code>Map</code> of IDs to numeric timestamps.
<code>recordLocal(dilemmaId)</code>	Adds a timestamp only if that ID is not already present.
<code>loadRemote(uid)</code>	Reads all private history documents and maps Firestore timestamps to milliseconds.
<code>recordRemote(uid, dilemmaId, known)</code>	Performs the idempotent remote write, using the dilemma ID as the document ID.
<code>mergeLocalInto(uid, known)</code>	Copies local-only entries into the signed-in account, updates the known map, and removes the local key.

8.3 js/selection.js

The selection functions are pure: they do not touch the DOM, Firebase, or storage. All random behavior accepts an optional random function, which lets tests inject predictable values.

Function	Behavior
<code>shuffle(items, random)</code>	Returns a copied array shuffled with Fisher-Yates; the input is not mutated.
<code>unseen(catalogIds, history)</code>	Filters IDs absent from the history map.
<code>pickSingleplayerQueue(catalogIds, history, size, random)</code>	Shuffles unseen IDs first, then appends seen IDs ordered by oldest timestamp until the requested size is filled.

Function	Behavior
<code>pickCandidates(catalogIds, size, random)</code>	Returns a random, non-repeating prefix no longer than the catalog.
<code>chooseDilemmaIds(candidates, counts, needed, random)</code>	Randomly breaks ties after sorting by count, takes the least-seen candidates, and cycles the chosen list if needed.

The core singleplayer policy is compact enough to show directly:

Listing 5: Repeat avoidance with a graceful fallback

```

1 export function pickSingleplayerQueue(catalogIds, history, size, random = Math.random) {
2   const fresh = shuffle(unseen(catalogIds, history), random);
3   if (fresh.length >= size) return fresh.slice(0, size);
4   const stale = catalogIds
5     .filter((id) => history.has(id))
6     .sort((a, b) => history.get(a) - history.get(b));
7   return fresh.concat(stale).slice(0, size);
8 }

```

8.4 js/scoring.js

Function	Behavior
<code>tallyChoices(choices)</code>	Ignores malformed choices and returns a <code>Map</code> from integer option index to count.
<code>pointsForChoice(optionIndex, choices)</code>	Returns zero for invalid/empty input; otherwise returns one for the unique top option or for every option when the top count is tied.

9 Function reference: tools, rules, and tests

9.1 tools/write-config.mjs

Function	Behavior
<code>parseEnv(text)</code>	Parses non-empty, non-comment <code>KEY=VALUE</code> lines, removes one matching pair of quotes, and returns a key/value object.
top-level generator	Maps six expected environment keys into Firebase config fields, substitutes placeholders for missing values, writes <code>js/config.js</code> , and reports whether configuration is complete.

The generated config is intentionally not treated as a secret. Firebase web configuration is visible to browsers; authorization is enforced by `firestore.rules`. `isConfigured` is derived from whether the API key still begins with `YOUR_`.

9.2 Catalog generator

Function	Behavior
<code>entry(theme, text, options, citation)</code>	Creates a catalog entry with a blank ID, which <code>main</code> fills in.
<code>reddit_source(title, path)</code>	Builds community-inspiration metadata with the standard subreddit URL prefix.
<code>main()</code>	Assigns sequential IDs, checks option count and uniqueness, checks duplicate themes/text, requires source URLs, writes formatted UTF-8 JSON, and prints the count.

The Python file is the authoring source for the catalog. The generated JSON is what the browser loads. In the current snapshot, `main` would assign IDs from `dilemma-0001` through `dilemma-0040`.

9.3 firestore.rules

Rules functions are small predicates used to compose access checks:

Function	Behavior
<code>signedIn()</code>	True when <code>request.auth</code> exists.
<code>roomData(roomId)</code>	Reads the room document and returns its data for rule evaluation.
<code>isHost(roomId)</code>	Requires sign-in and compares the room host UID with the caller UID.
<code>isPlayer(roomId)</code>	Requires sign-in and checks for the caller's player document.
<code>roundPhase(roomId, roundId)</code>	Reads a round's <code>phase</code> , used to gate choices.

9.4 Tests and test helpers

The tests are executable documentation of the intended boundaries:

- `test/scoring.test.mjs` covers counts, a unique majority, a three-way tie, invalid choices, and no choices. Its `choices(...optionIndexes)` helper makes player answer fixtures.
- `test/selection.test.mjs` validates catalog shape and source metadata, singleplayer freshness, least-recent fallback, room candidate ranking, cycling, and candidate uniqueness. It uses the catalog loaded with `readFileSync` and ID maps rather than stubs.
- `test/smoke.test.mjs` starts a small static server, launches headless Chrome, visits the actual page, and checks landing layout, theme persistence, singleplayer progression, source-backed options, sign-in routing, and console cleanliness. Its `text(selector)` helper reads visible text.
- `test/rules.test.mjs` runs against the Firestore emulator. Its `room`, `player`, `round`, `submitted`, `choice`, and `vote` helpers construct document references; `newRoom`, `newPlayer`, and `seedRoom` build repeatable fixtures.

The normal test command runs configuration generation plus the scoring, selection, and smoke suites. The rules suite is separate because it needs the Firestore emulator.

10 Markup and styling

10.1 HTML view structure

`index.html` is intentionally serverless and framework-free. The body starts with `data-view="home"`; a header contains sign-in and theme controls; the main area contains four mutually exclusive sections:

1. `view-home`: the title and Singleplayer/Play actions.
2. `view-auth`: password, sign-up, email-link, and sign-out controls.
3. `view-rooms`: create-room settings and join-by-code form.
4. `view-room`: room header, dynamic room body, error area, and leave control.
5. `view-single`: one prompt, source line, option list, and next button.

The five items above include the main home plus four content sections; the last four are the navigable work views. Dynamic content is inserted through DOM creation helpers rather than string HTML, which avoids accidental interpretation of catalog text.

10.2 CSS behavior

The stylesheet has no functions in the programming-language sense. Its main mechanisms are:

- root custom properties for foreground, background, muted, line, and soft-panel colors;
- a dark-theme override under `html[data-theme="dark"]`;
- a centered, narrow main column with a special vertically centered home layout;
- full-width, keyboard-visible option cards with hidden radio inputs;
- a 32 rem breakpoint that stacks the join form; and
- a reduced-motion media query that disables transitions when requested.

11 Build, run, and verification

11.1 Configuration

The repository keeps Firebase values in `.env` locally. Running `npm run config` creates the ignored browser module `js/config.js`. If values are missing, placeholders are written and the UI explains that Firebase is not configured; singleplayer still works.

11.2 Commands

Listing 6: Repository commands defined in `package.json`

```
1 npm run config      # Generate js/config.js from .env
2 npm start           # Serve the static app on port 5173
3 npm test            # Config plus scoring, selection, and smoke tests
4 npm run test:rules  # Firestore emulator rules tests
```

The application is a static site, so `npm start` only needs a local HTTP server. ES module imports and the catalog fetch are expected to run over HTTP rather than from a `file:` URL.

11.3 Verification checklist

Done	Behavior to verify
☐	Catalog loads, every entry has a stable ID, at least two unique options, and a source URL.
☐	Singleplayer records shown IDs privately, prefers unseen prompts, and falls back to least recently seen prompts.
☐	Sign-up, password sign-in, sign-out, email-link send, and email-link completion work with configured Firebase.
☐	A host can create a room, choose 1–20 rounds, and start only with at least two players.
☐	Players can join by four-character code and see the synchronized lobby.
☐	Selection shares only per-candidate counts, not personal history.
☐	Choices remain private until results; each player can submit only once.
☐	Unique majority scoring and tied-majority scoring match the rules.
☐	The host alone advances rounds and finishes the game; the scoreboard is cumulative.
☐	Firestore rules prevent cross-user history access, cross-player edits, invalid option indices, standalone votes, and non-host room control.

12 Notable implementation decisions

Copied round content

A round stores its prompt and option array when started. This protects a live game from later catalog edits and lets all players render the same content.

Client-side derived scoring

The result is derived from the visible choice set, while the write authority is restricted to each player's own score document. The rule is deterministic, and the transaction makes retries safe.

Privacy-preserving repeat avoidance

Players contribute counts rather than histories. The host gets enough information to rank candidates but cannot read another player's private history.

Idempotent progress

Remote history uses stable document IDs, choices are one-time creates, and score writes rebuild the total from per-round values. These choices make reconnects less dangerous.

Graceful offline/configured boundary

Firebase is optional for the singleplayer path. Missing config is surfaced plainly rather than causing the whole page to fail.

Small dependency surface

There is no frontend framework, bundler, server API, or runtime content scraper. The browser, Firebase Auth, Firestore, and static catalog are the whole product surface.

13 Summary

At the highest level, *dilemma* is a four-part pipeline:

1. **Content:** Python authoring data becomes a source-backed JSON catalog.
2. **Selection:** local history and shared candidate counts reduce unwanted repeats.
3. **Play:** the browser renders a small state machine backed by live Firestore snapshots.
4. **Trust:** Firestore rules enforce identity, host, membership, phase, field, and one-time-write constraints.

The implementation stays understandable because the UI coordinator is the only place that knows about DOM elements, pure game rules live in small modules, Firestore access is centralized, and the rules file mirrors the critical client assumptions. The main gap between the product note and the checked-in implementation is catalog size: the repository currently contains 40 entries, while the note describes 53. Everything else in this guide is based on the files present in the snapshot documented here.